Introduction to IPS and Brief Overview

D. B. Batchelor

1-6-2025

Introduction to IPS

The Integrated Plasma Simulator (IPS) is a computational framework that permits serial and or parallel simulation codes to function inter-operably on the most advanced massively-parallel computers so as to provide a flexible capability for integrated, multi-physics simulation. It was developed under the SWIM project in the DOE Scientific Advancement through Advanced Computation (SciDAC) program, jointly funded by the DOE offices of Fusion Energy and Advanced Computing Research. Afterwards it was supported by the AToM SciDAC project. Continued development and maintenance of the IPS is presently (2025) supported under the FREDA SciDAC project. The IPS has been extensively tested, has been installed on numerous computers around the world, and has demonstrated use of supercomputing resources in executing simulations comprised of a variety of different workflows. These range in complexity from simple time stepping simulations to complex event-driven workflows. IPS is implemented in framework/component architecture. It is not in any way restricted to plasma, or even to physics applications, and has found applications in several other fields, although many plasma physics related components are available for fusion applications. The IPS could reasonably be thought of as the Integrated *Process* Simulator. A number of people have contributed to development of the IPS, but the constant throughout the nearly 20 years since its inception has been Wael Elwasif of ORNL. He is THE authority on IPS.

IPS workflow components are typically independent codes, not originally developed with the intention of coupling to other components in an integrated way. The philosophy of IPS is to not require any modification of the component codes. In particular no modifications of the file I/O structure is required. The physics codes are 'wrapped' with interface coding (component-wrappers) that allows them to communicate and to inter-operate with the IPS framework and with other components.

The IPS framework itself is implemented in Python. It provides basic services needed by the components through a set of "managers". The Configuration Manager assembles and connects the needed components as specified in a simulation configuration file. The Task Manager manages the execution of the physics codes, which may be massively parallel. The Data Manager moves and archives the component input/output and simulation state files to make them available to the physics codes at each invocation. The Resource Manager efficiently manages access to compute resources for concurrently running processes. The Event Manager provides asynchronous publish/subscribe data exchange in a running simulation.

The IPS is designed primarily for use in a batch-processing environment, with a batch job typically comprising a single invocation of the framework, calling the individual component codes many times as the simulation progresses. IPS simulations are typically orchestrated by a "driver" component, although simulation-controlling logic can also be built into individual component-wrappers. The driver and component-wrapper interfaces are written in Python, and therefore can implement arbitrarily complex or conditional

Deleted: (Draft
Deleted: 4-2
Deleted: 2
Deleted: 19
Deleted: 6-2018
Deleted:)
Deleted:
Deleted: 19
Deleted: AToM
Deleted: is currently running
Deleted:
Deleted:
Deleted: ¶
Deleted: , i
Deleted:
Deleted:
Deleted:

workflows – essentially anything that is programmable. A simulation configuration file specifies which components make up any given IPS workflow. The config file allows considerable flexibility in the location of data and codes, and in details of component code behavior. Evolving simulation data that must be communicated between components as the simulation proceeds are contained in a set of files specified as *state* files, which are managed by the framework. The framework generates a directory structure for the workflow. It produces sub-directories for simulation results, restart data, progress logging, as well as individual work directories in which each component runs.

The IPS supports four levels of parallelism. The physics codes can be massively-parallel. Component can launch concurrent tasks, for example covering multiple flux surfaces or toroidal mode numbers. Multiple components can run concurrently. The IPS framework can manage multiple concurrently running simulations with a single batch submission. Allocated resources are efficiently managed using a task pool.

Although new capabilities have been added over the years, the IPS has remained stable and backward compatible. However, in 2020 a new version of the IPS was released in which the package install is now managed with python setuptools, and the IPS framework is installed into a Conda Python environment as a package called 'ipsframework'. This did require changes to the user installation procedure and some minor reworking of the IPS component python wrapper codes.

Deleted: The

Deleted: (e.g., ips.config)

Deleted:

Deleted: containing

Some IPS details

Simulation State

In order to interact, codes must exchange data. In the IPS the exchange is accomplished by specifying a set of files, known as STATE files, containing data describing the current state of the simulation. These are moved between components by the framework before and after code executions. Such STATE files are specified in the simulation configuration file. The framework maintains a *state* work directory containing the current set of state files, which are updated after each component execution.

Deleted: , which

Deleted: These

In general the output file of one component code will not be directly usable as input to another. It is a responsibility of the component-wrapper scripts to process the data in state files into the form needed by that component. See discussion below. In several cases where all, or many, components require the same data, it has proved beneficial to develop a common data format and a single file which multiple components use. This reduces the number of conversion algorithms required and increases the probability that components are actually using the same data values. For fusion applications the SWIM project developed an extensive system called the "Plasma State" that supports a data structure containing core plasma data and system description data relevant to tokamak devices. The SWIM Plasma State system provides many useful functions such as multiple instances, loading, saving, interpolation, copying, and modification-tracking that greatly assist in maintaining consistency of data across components. It is supported but not required by the IPS.

Configuration file

The makeup of the simulation is specified in a simulation configuration file. The configuration file, which is read by the IPS framework at startup, contains two kinds of parameters – global parameters that apply to the simulation as a whole and component specific parameters that are private to the individual components. Examples of global configuration parameters are [PORTS] NAMES, SIM_ROOT, and TIME_LOOP. NAMES is a list of abstract names of the components (or PORTS) to be used in the simulation. Each named PORT must have a section in the configuration file giving information about that PORT. SIM_ROOT, is the path to the root directory where the simulation will be run and the data collected. Typically the simulation root will the directory where the batchscript and configuration file reside, but it could be anywhere. TIME_LOOP is an iteration structure which could define a time stepping process, or it could be any appropriate iteraton index. Examples of component configuration parameters are the paths to the python wrapper codes that interface the framework to the physics codes, and paths to the executable of the physics codes. The user is free to define other parameters, either global or component specific, although he/she is responsible for programming the retrieval and behavior of user defined parameters. Much more information about configuration files is available in [Integrated Plasma Simulator (IPS) Documentation, Release 2.1, October, 18, 2011].

Components

IPS component-wrapper python scripts provide the interface between the IPS framework and the component codes. Component-wrappers are python classes, which must inherit from the IPS class *Component*. Components are required to provide three functions, *init*, *step*, and *finalize*. Many also implement *checkpoint* and *restart* functions to use the IPS built-in checkpoint/restart capability. Users may also provide other component-wrapper functions, which for example drivers might use, but the framework makes no reference to them.

There are two distinguished PORTS called by the framework itself – INIT, and DRIVER. The workflow INIT performs any initializations needed before the components perform their individual *init* functions. The DRIVER calls the *init* functions of the other component-wrappers and starts the workflow execution. It usually provides the logic for the progression of the workflow by calling the *step* functions of the components. It calls the *finalize* function for each component-wrapper at the end of the workflow. The purpose of the component-wrapper *init* functions is to do any work needed to prepare the component for the beginning of the simulation. In particular it must generate any initial data that goes into the state files and must update the state work directory using framework services. The *finalize* functions can perform any needed final calculations or clean up any mess left behind, but most often do nothing.

Physics component-wrapper scripts have three primary responsibilities as the simulation progresses:

1) To generate standard input files for the implementing component codes. Input data for the codes most often is a combination of code specific data derived from standard, template, input files of the given code, evolving data derived from the simulation state files, and perhaps parameters obtained from the configuration file. This is accomplished by calling on framework services to move input files and simulation state files into the work directory, then merging the data to generate

Deleted: simulation

Deleted: and

Deleted: The file specifies a set of "PORT"s, which point to the implementation of each component of the workflow….

Deleted: Required global parameters include: IPS

Deleted: which

Deleted: IPS framework itself,

Deleted: and NAMES, which is a list of abstract names of the components (or PORTS) to be used in the simulation. There are also required component parameters such as: SCRIPT, which is the path to the wrapper script that implements the component.

Deleted:

Commented [GDL1]: URL for this?

Deleted: .

Deleted: ,

Deleted: , i

Deleted: , and

Deleted: i

updated standard code input files. It may involve invoking other codes in the process.

2) To call on framework services to assemble the computer resources needed and launch the physics code execution using the appropriate parallel protocol for the platform in use.
3) To process the code output to the form appropriate to write into the STATE files, for example converting to a common data format if one is specified. It should then call on framework services to update the collection of state files containing new data, and to archive any other code specific output files that are to be kept as results.

Component-wrapper scripts are free to provide other functions to be called by the driver, or to launch other physics codes needed to fulfill the functionality of the component. Component-wrappers can get and employ global or component specific configuration parameters. Some component-wrappers have been developed which have extensive internal logic or which react to signals directly from other component-wrappers sent via the framework publish/subscribe event service.

Other IPS features

The IPS provides many other functions and services. We describe just a few here.

Framework event tracking – The framework generates a unique, human readable (although long) run identifier for each IPS workflow execution, which allows some level of provenance tracking. The identifier is available to component-wrappers as a global configuration parameter, RUNID. The IPS produces an event log file identified by the RUNID that contains a record of every action the framework takes throughout the run. This data is valuable for following the progress of the workflow, debugging, and chasing down timing issues. Periodically and at the end of the run the event log file is processed to an html file that can easily be displayed in a web browser. These two files reside in the */simulation_log/* subdirectory. In addition the user can specify a directory outside the IPS generated workflow directory where the html files are sent. This way html log files for all of the user's IPS runs can be collected for reference. At NERSC, in the simulation config file one can specify that this tracking data be sent to a web accessible PORTAL which can be viewed from the user's browser.

Data Monitoring – For workflows that use the Plasma State, an IPS component called MONITOR is available which serves two functions. At the end of each step, it extracts and processes selected plasma data from the current plasma state file (which itself is a time snapshot) and aggregates the data from multiple steps into time series which are written to a monitor netcdf file. The current monitor file is copied to the/www/ directory each step. For any simulation the user can specify in the configuration file one or more state files to also be copied to the /www/ directory at the end of each step. In addition, a capability (called PCMF.py) is provided to do a graphic dump of all the time series and profiles in the monitor file into a pdf file.

Work-flows for massive numbers of serial executions – Large numbers of Python executions cause parallel file system issues. So, a capability has been developed for hierarchical work-flow that eliminates the problem using containers (shifter). The system

Deleted: physics

Deleted: for

Deleted:

Deleted:

Deleted: ¶

Formatted: Font: Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold

has been used for several applications including for training machine learning algorithms (see https://ieeexplore.ieee.org/abstract/document/9297046)

Nested workflows – Also developed is capability to use complete IPS simulations as component sub-workflows in a 'parent' workflow. These can be nested to any depth. Such techniques have been used in Plasma-Surface-Interaction (PSI) modeling – SOLPS, HPIC, F-TRYDYN, GITR, Xolotl

(see https://ieeexplore.ieee.org/abstract/document/9035247)

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Bold

Deleted: ¶

What do you need to run an IPS Simulation?

1) You need access to the IPS. This is easily downloaded from the github repo (see Getting Started, below).

Formatted: Font: Not Bold

2) You need executables for the physics codes you want to run, plus wrapper codes to connect to IPS and to access the state files you want to get data from and put results into.

3) You need a starting set of input files for all the physics codes you are using. Typically, the starting files act as templates which are modified as the simulation progresses and the code inputs change.

4) You need a simulation configuration file.

5) You need a batch script file. This is a standard batch queue submission script for the particular operating system (slurm at NERSC). It points to the particular simulation configuration file and invokes the IPS.

6) You need a simulation root directory containing the batch script and the simulation configuration file. Depending on details other files may be needed, such as environment shell scripts.

Formatted: Font: 12 pt, Not Bold

What do you get from an IPS simulation?

The IPS framework populates the simulation root directory with work subdirectories for each component and with various directories for simulation results and archive.

0) RUNID – A text file containing the unique identifier for the run.

1) simulation_setup – Directory containing copies of the python initializer, drive, and component codes plus subdirectories containing the initial input files for all components

2) work – Directory containing subdirectories for the initializer, driver, and all components The components run in these directories which therefore have all input and output files, any needed state files and code log files.

3) simulation_results – Contains subdirectories for initializer, driver, state and all components. Each of these subdirectories contains a subdirectory for each time step (or iteration) with the files designated in the configuration file as the outputs of each component. These are the results you wanted.

4)simulation_log – Contains files <RUNID>.eventlog, <RUNID>.html, and <RUNID>.json which record all of the actions that the framework takes during the run.

5) restsart – Contains data needed to restart the simulation from a given checkpoint. The checkpoint schedule and the specific restart data to be captured is specified in the configuration file.

6) Various log files for stdout and stderr

What you don't get is an aggregation of data throughout the simulation unless you use the PLASMA STATE and the MONITOR component, or you program it yourself. The data in the simulation_results directory is the collection of the individual time slices, not time series. Considering the wide variety of outputs that might be produced, and that different users might want, it is not feasible to produce a general analysis or graphics capability.

Getting started with IPS

The place to start with IPS is to download the framework, get some simple example simulations, and just run them. The first stop should be

https://ips-framework.readthedocs.io/en/latest/

which gives instructions for downloading ips and setting up the needed Conda environment. Instructions for cloning the needed wrapper and examples github repos can be found at https://github.com/ORNL-Fusion/ips-examples. The relevant URLs are:

git clone https://github.com/ORNL-Fusion/ips-wrappers.git
git clone https://github.com/ORNL-Fusion/ips-examples.git

There are two examples that are particularly appropriate for getting started – hello-world and ABC_example.

hello-world example

Hello_world is (almost) the simplest possible IPS run, exercising two components DRIVER, hello_driver.py, and WORKER, hello_worker.py. We say "almost" because it would actually be possible to have an IPS simulation with only a driver and no components. Hello world does not use any STATE files or use any physics components. There is one external connection, to an environment file (env.ips) that is sourced from $IPS_DIR/ips-examples/env.ips in the present setup.

ABC_example

The ABC simulation is a somewhat more realistic example of IPS usage. It is completely written in python and has no dependencies other than IPS. As such it should run about anywhere, at least it runs on Macs and at NERSC. The input and output files are all human readable text. It largely has the structure of a real simulation in that it has three components that interact. The "simulation" solves a coupled set of 2 ODEs by bonehead step-by-step integration.

$$x = Ax + Bxy$$
$$y = Cy + Dxy$$

Deleted: and

Deleted:

Deleted: For

Deleted: To give an idea of what is involved,convenience the process as of 4/2018 is repeated here, but it is (subject to change so the instructions at github should be considered definitive.).

Deleted: Install the IPS¶
1) Create an IPS directory and clone the IPS-framework, wrappers, and examples repos.To clone the repositories create a directory where your IPS installation will live, e.g.¶
mkdir IPS¶
cd IPS¶
git clone https://github.com/HPC-SimTools/IPS-framework.git ips-framework¶

Moved (insertion) [2]

Deleted: three

Deleted: ,

Deleted: , and sequential_model_simulation (not cleaned up yet).

Deleted: 2) Export the IPS_DIR environment variable¶

Formatted: Font: +Body (Cambria)

Moved up [2]: There are three examples that are particularly appropriate for getting started – hello-world, ABC_example, and sequential_model_simulation (not cleaned up yet).¶

Deleted: mkdir IPS¶
cd into it and clone the IPS framework and wrappers/examples repos.¶
cd IPS¶
git clone https://github.com/HPC-SimTools/IPS-framework.git ips-framework¶
git clone https://github.com/ORNL-Fusion/ips-wrappers.git¶
git clone https://github.com/ORNL-Fusion/ips-examples.git¶
To run one of the examples, such as hello-world, if you are not already there, cd to the IPS top level directory e.g. /IPS/ then¶
export IPS_DIR=${PWD}export IPS_DIR=${PWD}¶
source ips-wrappers/env.ips¶
¶
¶
3) Add this to your .bashrc (or otherwise so it's there next time you open a shell). Note: Adapt for csh or otherwise.then cd into the particular example directory and run¶
¶
… [1]

Commented [GDL2]: We should update the README.md file in the ips-examples repo of this example to have useful documentation also.

Deleted: ¶

Deleted: ¶

There are 3 “physics” codes: *X_dot_code.py, Y_dot_code.py*, and *integrator.py*. Component wrapper scripts drive these “physics” codes: *A_component.py, B_component.py*, and *C_component.py* respectively. In addition there are simulation initializer and driver components: *basic_init.py* and *basic_driver.py*. The *basic_init.py* and *basic_driver.py* components are very generic and might well be usable directly in other simple workflows.

The codes are bundled in the *_exec* directory and all of the input data is in the *‘_input’* directory. Each code has a template input file and produces one output file. The STATE files consist of output files from the A and B components and a common state.dat file. It also will send the eventlog html file to a www directory. So it pretty much has the structure of a real simulation.

Typically a real simulation would be run as a batch job, submitted via a batch script. In trying to make this like a real simulation running as a batch job, a batch script for Perlmutter and a shell script for Mac are provided. The command lines to run from the /ips-examples/ABC_example/ directory are:

On Perlmutter – sbatch Perl_run

and on Mac – ./Mac_run

This example also demonstrates simulation restart which is invoked with the Perl_restart and Mac_restart scripts.

Other examples and workflows

There are a number of other examples available that use real physics codes. Also available are a number of simulation workflows and simulation configuration files which might fit the user’s needs with no modification, other than change of input parameters. However these will generally require access to prebuilt codes and wrappers that are not provided with the installation above.

Plasma physics, wrappers, and maintained installations

The hello-world and ABC example are entirely Python and require no compilation or other build steps. However most of the applications of IPS involve significant physics codes, primarily plasma physics, which do require compilation and building. Up to now IPS has not attempted to distribute the physics codes themselves, but relies on code developers to maintain and distribute them. The IPS does maintain a collection of interface component-wrappers to allow many physics codes to be used in the IPS (i.e. the contents of the ORNL-Fusion/ips-wrappers.git repo cloned above). Many of the wrappers use fortran helper-codes that interact with the Plasma State module for preparing physics code input files and processing physics code output data to put into the plasma state. The fortran helpers require compilation and building, so such component subdirectories contain makefiles. It is not always a trivial matter to build these wrappers, particularly ones that couple to the SWIM Plasma State, which itself is not simple to build. In the past, to obviate the need for a user to go through this process, the IPS team has maintained pre-built IPS installations of wrappers, along with collections of built physics codes at several computer centers. Such installations were made available at many locations around the world, although the

Deleted: *codes*

Deleted:

Deleted: is

Deleted:

Deleted: Edison

Deleted: The exporting of IPS_DIR and sourcing of ips.env are included in these scripts so these two steps described above can be omitted.

Deleted: , on

Deleted: Edison

Deleted: ,¶

Deleted: Edison

Deleted: ,¶

Deleted: Edison

Deleted: ¶
sequential_model_simulation¶
(Under construction)¶

Deleted: ¶

Deleted: _

Deleted: To

Deleted: s

Deleted: have been

principal one active now is at NERSC. If a user wishes to use the existing IPS component base it is easiest to do at NERSC.

For a green-field development, in which the user has in hand the physics codes to be coupled, and there is no need for any of the compiled IPS components (some of the IPS components are entirely Python and require no compilation), the IPS system cloned above can be used locally. The user will just need to provide the interface wrappers for his particular physics codes.

Developing new component wrappers

The required functions of component-wrapper scripts are all the same, as described above. As such, much of the coding is identical among the wrappers, particularly the code needed for communicating with the IPS framework. The logic may differ between components but that is generally easy to program in Python. The biggest jobs are translating the state data into the proper format for the physics code's input file/files, and putting data from the code's output files back into the state files for use by other components. The advice is, before launching into a new component development, talk to someone on the IPS team who has done it. It is essentially never necessary to start from scratch.

There already exist a wide variety of driver components and physics components that can serve as templates for new development. In addition, there are drivers, initializers and components that are quite generic and might be directly useable for new workflows, thereby obviating the need for any python programming at all. Specifically 'basic_init.py', 'basic_driver.py, and generic 'component.py'. Are completely programable within the configuration file and can implement simple workflows directly.

Deleted: s

Deleted: are

Deleted: , General Atomics, and ORNL.

Deleted: it through one of the maintained installations

Deleted: , although other installations can be made, and we anticipate making it easier in the future.

Deleted: s

Deleted: e

Deleted: between

Deleted: that

Deleted: assembling

Deleted: to produce

Moved (insertion) [1]

Deleted: ↵

Deleted: Some

Deleted: of the

Deleted: and

Moved up [1]: The advice is, before launching into a new component development, talk to someone on the IPS team who has done it. It is essentially never necessary to start from scratch.↵

Deleted: .

Resources

Web guide for IPS: https://ips-framework.readthedocs.io/en/latest/
This is the best place to start

IPS git repo: https://github.com/HPC-SimTools/IPS-framework

IPS wrappers repo: https://github.com/ORNL-Fusion/ips-wrappers

IPS examples repo: https://github.com/ORNL-Fusion/ips-examples

Introduction to and Brief Overview of IPS.pdf
This document: https://github.com/ORNL-Fusion/ips-examples/blob/master/Introduction%20to%20and%20Brief%20Overview%20of%20IPS.pdf

IntegratedPlasmaSimulatorIPS.pdf : Relatively detailed write–up as of 2011

IPS portal: https://ips-framework.readthedocs.io/en/latest/intro.html

Formatted: Font: 14 pt, Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: No bullets or numbering
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: 11 pt
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold